

Contents

Memory retrieval white paper — Version 2 plan 1

1. What changed since V1 was written 1

2. Three findings that reshape the plan 2

2.1 The APM queue has ~48 problems of headroom, not ~360 2

2.2 Recall loss is structural, diagnosed, and non-randomly biased 3

2.3 Bitemporal as-of fails on two different routes, for two different reasons — and neither is XTDB 3

3. What V2 can and cannot claim 5

3.1 Off the table for V2 5

3.2 Newly on the table for V2 5

3.2.1 Raw job results — captured, and they were the deadline item 5

3.3 The “used ≠ used well” gap 5

4. Proposed V2 experiment programme 6

5. Proposed V2 identity 7

6. The V3 plan 7

6.1 What V3 is for 7

6.2 The critical path, in order 8

6.3 Instrumentation V3 requires (unrecoverable if missed) 8

6.4 The War Machine as a non-APM source — thin, and thin in a specific way . 9

6.4.1 Correction: the correct implementation is neither lane — it already exists 10

7. Division of labour and gates 10

8. Decisions taken, and what remains open 11

Settled by Joe, 2026-07-31 11

Still open 11

Memory retrieval white paper — Version 2 plan

Status: proposed, pending agreement with claude-9 and scoping by Joe **Opened:** 2026-07-31 (claude-2) **V1:** docs/retrieval-whitepaper.md — frozen, keep as Version 1 **Ledger:** docs/retrieval-evidence-ledger.md — chronology, unchanged **Supersedes for planning purposes:** E-memory-whitepaper-plan.md \$Experiment 0

1. What changed since V1 was written

V1 was written on 2026-07-30 against a 16-half receipt slice filtered to author=ground-control. An overnight run supervised by claude-9 has since completed. The live corpus is ~17× larger and was written under a different author, so V1’s headline channel figures describe a slice, not the system.

Measured by claude-2 against the live store on 2026-07-31, frozen as receipts-export-20260731-all-authors.edn and audited by observation_channel_audit.py (regression-checked: it still reproduces 16/14/2/2/1 on the 07-28 export):

	V1 (07-28 slice)	V2 (07-31, all authors)
Offered halves	14	129
Outcome halves	2	115
Outcome-half completion	14.3%	89.2%
Joined rows	2	114
Metric-bearing rows	1	20
Recall :ok	4 (29%)	47 (36%)
Recall empty	10 (71%)	82 (64%)
:timeout / :store-unavailable	2 / 0	46 / 12
Total surfaced ids	8	215
surfacing-via populated	0	60

	V1 (07-28 slice)	V2 (07-31, all authors)
inclusion-reasons	4 rows, 1 string	absent (superseded)

Two V1 predictions resolved.

- V1 §8.2.1 item 2 named outcome-half completion as “the single highest-leverage repair”. It went 14% → 89%. That repair has landed.
- V1 §8.2.1 item 1 warned that “a deployed writer is not a populated field” for surfacing-via. Verified populated on 60 rows. Closed.

One V1 assumption refuted. V1 §8.2.2 sized draft 2 against “489 dispatches” of headroom. That headroom does not exist (§3.1).

2. Three findings that reshape the plan

2.1 The APM queue has ~48 problems of headroom, not ~360

Three different counts of “problems with Lean” were in circulation — 89 (my first pass, from `processed_level`), 135 (the sanctioned counter), and 145 (Main.lean files on disk). claude-9 flagged the gap as load-bearing and declined to lock any of them in. That was right, and the resolution is that **none of the three was the number we wanted**.

Classifying all 145 `problems/*/lean/Main.lean` files by whether they contain a declaration and ≥ 8 non-blank lines:

	substantive	stub
In manifest’s Lean-bearing set	52	24
Not in it	69	0
	121	24

The manifest’s `processed_level` fails in *both* directions: it admits 24 scaffold-only stubs and misses 69 substantive files. (claude-9’s hypothesis was that the gap was stubs the manifest correctly excluded; it is the opposite.)

The addressable set is 121 substantive Lean problems. Normalising the receipts’ `:problem` slugs to APM ids — three slug shapes occur (`a92J05-...`, `hard-problems-a94j07-...`, and 15 named construction targets such as `rouche-root-count-transfer` that are not APM rows at all) — yields 74 receipted ids, 73 of them substantive. This reproduces claude-9’s independent count of 73 exactly, which is the cross-check that the normalisation is sound.

121 substantive – 73 receipted = ≈ 48 problems of headroom

Not ~ 16 (my first figure), not ~ 360 (V1’s assumption). ≈ 48 is roughly a 37% increase on the 129 dispatches in hand: **material for RQ-level statistics, nowhere near enough for per-coefficient promotion.**

claude-9 reports the run finished with 0 dispatchable rows and nothing in flight; the queue stands at 50 resolved / 27 blocked-and-diagnosed / 4 partial / 1 dependency-blocked. So the 48 are not queued work — reaching them is a scoping decision for Joe, not a resumption.

Two prerequisites gate the 48, and neither is imminent.

- *The recall-budget fix* (§6.3). It has **no owner and is not in progress**; claude-9 identified it and deliberately did not apply it, since it changes dispatch behaviour for every future run. It is Joe’s call, not imminent work. Crucially it must be **validated on 2-3 throwaway dispatches with a measured drop in :timeout rate before the 48 are committed** — “the fix landed” is not “recall completes”, and spending the last clean problems to buy a partial fix would be the worst available outcome.

- *A census step.* The queue covers 74 APM problems while 145 Main.lean exist on disk, so **71 problems have no queue row at all**. The headroom is not waiting to be dispatched; it must first be censused into rows (statement hints, :line, kind), and claude-9's session surfaced that step's own failure modes: stale hints silently repointing at a neighbouring target, empty :line lists throwing in the selector, two rows sharing one file. This work is **independent of the recall fix and can proceed in parallel**.

Consequence: V2 is built from the 129 dispatches in hand. The 48 are a V3 input, and worth dispatching only *after* the recall-budget fix (§2.2), because dispatching them now would reproduce the 45% infrastructure-loss rate on the last untouched problems we have.

2.2 Recall loss is structural, diagnosed, and non-randomly biased

This is V2's strongest new systems result, and it is a mechanism rather than a rate. Verified independently by claude-2:

- default-recall-timeout-ms is **30 000 ms, a total budget** (dispatch_with_recall.clj:19, documented as such at line 136).
- evidence/text-search costs **9-16 s per query** on a healthy store (claude-9, receipt 4f7eeadd).
- Recall issues **one query per subject**, with 6-12 subjects per row.

So recall **structurally cannot complete** for any row carrying more than about two common subject terms. 46 of 129 dispatches timed out; with 12 :store-unavailable, **58 of the 82 empty results (71%) are operational failures rather than retrieval misses** — i.e. ~45% of all dispatches lose recall to infrastructure.

The bias is **not random**. claude-9 measured clustering by term commonality rather than subject count: construction-deriv timed out on 6 subjects (deriv, zero, isopen, differentiable) while a97A08 succeeded on 6 (count, outer, roots, filter, card, inner). It falls hardest on analysis-heavy rows whose subject terms are common mathematical vocabulary. A 04:40 store restart halved the tail (29.9 s → 14.4 s for deriv) without fixing it.

This directly vindicates V1 §5.5(i) — *missing observations are not negative observations* — and upgrades it from a caution to a measured, directional bias that any analysis over this corpus must carry.

2.3 Bitemporal as-of fails on two different routes, for two different reasons — and neither is XTDB

This section was rewritten 2026-07-31 after tracing the defect to source. The earlier conclusion — “bitemporality is broken, Experiment 0 is dead” — was drawn from testing one endpoint and was too broad in one direction and too pessimistic in another.

claude-9 and claude-2 independently observed that system-as-of had no effect on GET /api/alpha/evidence:

```
GET /api/alpha/evidence?type=pattern-outcome&limit=3           → 17441 bytes
GET /api/alpha/evidence?...&system-as-of=2026-07-31T04:00:00Z → 17441 bytes
GET /api/alpha/evidence?...&system-as-of=2020-01-01T00:00:00Z → 17441 bytes
```

Byte-identical, no error. Both of us then generalised from this to “the store ignores as-of”. **That generalisation is wrong.** Tracing to source:

Defect 1 — the evidence route never reads the parameter. evidence-route (futonlb_server.clj:382-426) dispatches to ev/query-evidence-response, and futonlb_evidence.clj contains **zero** occurrences of as-of, basis, or snapshot. The parameter is dropped in futonlb's own handler and never reaches XTDB. Silent, undocumented, and the caller receives present-time data believing it is historical.

Defect 2 — the projection route implements as-of correctly, then refuses to return it. `memory-projection-route` (`futonlb_server.clj:582–604`) *does* parse both parameters, and its own comment states “Explicit bitemporal projection still reaches XTDB”. Tested:

request	result
no as-of, limit 10	200, 22 973 bytes, 10 memories
system-as-of=2020-01-01	200, 571 bytes, 0 memories ✓ correct
system-as-of=2026-07-29 (any in-range value, any limit 3/5/10)	400 :memory-projection-result-bound-exceeded

The 2020 case is the important one: it returns **empty**, which is the correct answer for a store that did not exist then. **Bitemporality on this route genuinely reaches the engine.**

The 400 is ours. `futonlb_graph.clj:942–949` fetches (`inc raw-limit`) rows and throws if the count exceeds `raw-limit`:

```
(list 'limit (inc raw-limit))
...
(when (> (count selected+) raw-limit)
  (throw (gates/layered-error 4 :memory-projection-result-bound-exceeded ...)))
```

Under an as-of query the underlying scan returns **every historical version** of each edge, so the row count always exceeds the limit and the guard always fires. The guard cannot distinguish “too many distinct results” from “the same results, many versions” — it is a truncation check that predates bitemporal use and is blind to version history. Lowering the limit does not help; it lowers the threshold in step.

Consequence for the plan, in both directions.

- *Do not report this to JUXT as an XTDB bug.* Both defects are `futonlb` application-layer faults with named source lines. Taking them to James Henderson as XTDB issues would be wrong on the facts. (There may be a legitimate adjacent question about XTDB history-retention semantics, but nothing measured here establishes one.)
- *Experiment 0 is less dead than V1 and the first draft of this plan said.* The projection path — which is what `dispatch_with_recall.clj:376–388` actually uses — reaches XTDB bitemporally. If the result-bound guard is made version-aware (dedupe by entity id before the count, or count distinct entities), dispatch-time **graph** state may become reconstructible without per-problem snapshots. That is a plausible partial rescue, and it is a one-file change in our own code rather than a dependency on anyone else.

Claim discipline. What is verified: the parameter reaches the engine on the projection route (2020 → empty), and the bound guard blocks in-range queries. What is **not** verified: that XTDB retains enough history to reconstruct 07-25 state, or that valid-time versus system-time semantics give what Experiment 0 needs. Those are the next tests, not established results, and V2 must not claim the rescue until they run.

Partial rescue on the query side. `subjects_for(row)` is deterministic, so the terms issued per dispatch *can* be recovered and paired with surfaced-ids, giving (query → returned) pairs — more than the receipts alone carry, and enough to characterise which queries starved. Two caveats that must travel with any such reconstruction:

- The rows have been **edited since dispatch** (hints refreshed, `:line` repointed), so recovery is approximate and biased toward the *original* dispatch rather than the current row.
- **57 rows carry duplicate keys**, and the parser keeps the **oldest** value. That happens to bias in the helpful direction here, but it is an accident of parser behaviour, not a designed guarantee, and it should be verified per-row rather than assumed.

This does **not** restore `recall@k` — nothing freezes what was *retrievable* — but it makes “which queries returned nothing, and did they share a term profile?” answerable, which is the evidence §2.2’s bias claim currently rests on indirectly.

3. What V2 can and cannot claim

3.1 Off the table for V2

- **Experiment 0 as specified** (frozen chronological benchmark with dispatch-time corpus state) — blocked by §2.3, unretrofitably for existing data.
- **recall@k / nDCG against a frozen corpus** — same cause.
- **Threshold B / per-coefficient promotion** — $n \geq 20$ *per coefficient* needs volume that §2.1 shows is not available from APM.

3.2 Newly on the table for V2

The overnight corpus turns out to carry a label set richer than V1 knew about. Across 255 receipts claude-9 reports used-ids 238, unused-ids 182, rejected-ids 126, **rejection-reasons 129**, retrieval-to-use-ms 126.

That is a **three-way used / unused / rejected label per surfaced id, with free-text grounds** — captured in production, at scale, without a labelling campaign. It substitutes for a meaningful part of what Experiment 0's relevance judgments were for. Sample quality is high ("the imported contraction theorem handles kernel integrability internally"), and a94j04 shows a distinct *considered-but-declined* category.

This is the single most valuable asset the overnight run produced, and V1 did not know it existed.

Two scoping constraints, from claude-9, to be stated in the paper rather than discovered by a reviewer:

1. *These are solver self-reports, not ground truth.* The taxonomy is of **stated grounds for declining**, not of actual irrelevance. Nothing validates a "rejected" label — a wrongly-rejected memory looks identical to a correctly-rejected one — and the reasons are produced by the same model that made the decision, so they are partly post-hoc rationalisation. The e9d008be case (§3.3) is the mirror image on the "used" side.
2. *It is one model.* All 129 come from codex runners (codex-7 54, codex-6 33 in claude-9's sample; runner-model :codex throughout). This is **codex's** rejection behaviour, not a general property of solvers, and it belongs in the title or first paragraph.

3.2.1 Raw job results — captured, and they were the deadline item

claude-9 identified the one item in this exchange with a clock on it: the runner's **full verbatim output** is retrievable from GET /api/alpha/invoke/jobs/<id> and is richer than the receipts — route, search audit across all three arms, error→fix log, and the memory-usage section with per-memory reasons *as written*. Retention is unknown, and an expired job is indistinguishable from a mistyped id through that endpoint.

Captured 2026-07-31 by claude-2 before proceeding: all 130 distinct job-ids from the frozen receipts export, into holes/labs/M-memory-retrieval/job-results-20260731/ — 130 files, 1.4 MB, 2.8 KB–87 KB, none truncated, zero fetch failures. (One file matches :error; inspection shows 21 occurrences inside the runner's own error→fix log, not a fetch fault.)

V2-3's $n=129$ now rests on disk rather than on server retention.

3.3 The "used ≠ used well" gap

claude-9 found a case (receipt e9d008be) where a memory was surfaced, correctly used, and still produced the wrong decision — it carried a fact without its consequent, and the run burned a slot on an unnecessary frontier. used-ids scores that as a success.

This extends V1 §2.3’s trust-boundary argument with a fifth boundary the paper does not currently name:

Boundary	Establishes
Review	the edge is well-formed
Warrant	evidence supports the content
Attribution	the solver says it used the memory
Load-bearing	the memory changed the outcome for the better
Witness	a third party confirms the result

V2 must either measure the load-bearing label or state explicitly that any reported use-rate overstates. Recommend: measure it on a subsample, state the gap in the abstract.

4. Proposed V2 experiment programme

All of these run against **data already in hand**. None waits on dispatch volume.

#	Experiment	Input	Gate	Owner
V2-1	Channel re-audit, before/after	done — observation-channel-audit-20260731.edn	none	claude-2 ✓
V2-2	Timeout mechanism + bias characterisation	129 offered halves, receipt 4f7eeadd	none	Codex
V2-3	Rejection-reason taxonomy	129 rejection-reasons + job-results-20260731/	code against claude-9’s pre-registered 5 categories first	Codex, claude-9 reviews
V2-4	Ψ-v2 replay at n=20	20 metric-bearing rows	report honestly, expect per-coefficient abstention	Codex
V2-5	D_state sweep at scale	73 problems w/ receipts (vs 2 in V1)	current graph, not dispatch-time — state the limitation	Codex
V2-6	Load-bearing subsample	~20 receipts, manual	claude-9 + claude-2 adjudicate	joint
V2-7	Floor sensitivity (multi-seed)	synthetic	none — carried over from V1 §8.2.2 E4	Codex
V2-8	:witness-status repair + re-audit	warrant_audit.py	repair must land first	Codex

V2-3 is the headline. A taxonomy of *why reviewed memories were declined by a working solver*, at n=129 with free-text grounds, is a contribution no other agent-memory paper has the data to make.

Pre-registered category set (claude-9, from supervising the run, recorded here *before* coding begins so it cannot be fitted to the argument). All five were observed live:

1. **Topical mismatch** — different subject entirely (“unrelated dimension and integral”, “unrelated problem class”).
2. **Scope mismatch** — right area, wrong sub-object (“concerns *parameterized* Laplace transforms”; “no *compact-support* Laplace transform here”).
3. **Subsumption** — relevant but already handled (“the imported contraction theorem handles kernel integrability internally”). A substantive mathematical judgement, not a topical one.
4. **Stage mismatch** — relevant, but to a *later* target (“relevant only to the later a.e. target”). The *considered-but-declined* category, and the one that evidences discrimination rather than matching.
5. **Precondition absent** — the memory’s trigger never fired (“no instance diamond arose”).

Coding protocol: code all 129 against these five first; record the unclassifiable residue honestly; only then decide whether to split or merge. A sixth category emerging *from the residue* is a finding. A sixth category emerging because five did not fit the argument is fitting, and the distinction is the reason the set is registered here.

V2-5 carries a mandatory caveat: without dispatch-time snapshots it measures the *current* graph, so it is a structural-sensitivity result, not a historical replay. Say so in the caption, not just the limitations section.

5. Proposed V2 identity

V1’s identity was “*architecture and its measurement instrument, with a pre-repair baseline.*” V2 should become:

A production agent-memory system measured end to end: what a warrant-disciplined retrieval loop actually delivers over 129 real theorem-proving dispatches, why 45% of its recalls never complete, and what a working solver says when it declines the memories it is offered.

That is honest, it is supported by data in hand, and it is more interesting than the ablation V1 was holding the paper open for. The hybrid-ablation question (V1 RQ1) moves explicitly to V3 with its blocker named (§6.1, V3-C1).

V2 is the artifact shown to Rob (Joe, 2026-07-31), conditional on carrying the V3 plan at §6. Two consequences for how it must be written: every claim has to stand without the ledger as context, since an external reader will not have it; and the codex-only scoping constraint (§3.2) belongs in the title or first paragraph, not the limitations section.

6. The V3 plan

Joe, 2026-07-31: V2 may be shown to Rob **provided it carries a plan for V3**. This section is therefore written to be read by an external reader, not just as an internal checklist. It states what V3 claims, what gates each claim, and in what order the work runs.

6.1 What V3 is for

V2 is an honest account of a system measured through an instrument that was itself partly broken. **V3 is the effectiveness paper V1 was holding itself open for:** does warrant-disciplined hybrid retrieval actually retrieve better, and do receipt-updated coefficients actually rank better?

Two claims, no more:

- **V3-C1 (hybrid).** Combined content and pattern retrieval improves relevant recall at a fixed budget over either route alone. *Gate:* the frozen benchmark of §4.2 — n queries over ≥ 15 theorem families, ≤ 3 per family, paired analysis on discordant pairs, n sized from a 10-12 query pilot measuring the discordance rate.
- **V3-C2 (adaptation).** Receipt-updated coefficients improve ranking on later observations against no-update and scalar-update baselines. *Gate:* $n \geq 20$ independently

witnessed outcomes **per coefficient**, evaluated chronologically rather than leave-one-out. Partial promotion — “top k coefficients promoted, tail abstained” — is a legitimate result, not a fudge; the harness’s per-coefficient abstention is designed for it.

If the data do not support a claim, V3 reports the null against the preregistered evaluation and does not re-cut the analysis. That rule is what V1’s §5.2 and §5.3 already demonstrate we will actually follow.

6.2 The critical path, in order

Each step gates the next. Steps 1–3 are prerequisites and none is currently in progress except where noted.

1. **Fix the recall budget** (*owner: claude-9, assigned by Joe 2026-07-31*). See §6.3. Until this lands, ~45% of any new dispatch is lost to infrastructure and the marginal problem is worth ~half a sample.
2. **Validate the fix on 2-3 throwaway dispatches** with a measured drop in `:timeout` rate — *before* committing the ~48 remaining clean problems. “The fix landed” is not “recall completes.”
3. **Census the 71 un-queued problems into rows** (statement hints, `:line`, `kind`). Independent of step 1, can run in parallel. Known failure modes: stale hints silently repointing at a neighbouring target, empty `:line` lists throwing in the selector, two rows sharing one file.
4. **Repair the projection result-bound guard** (§2.3, Defect 2) and re-test whether dispatch-time graph state is reconstructible. This is the cheap shot at Experiment 0 — a one-file change in our own code — and it should be attempted before anyone builds per-dispatch snapshotting.
5. **Dispatch the ~48** with instrumentation from §6.4 in place.
6. **Build the frozen benchmark**, then run V3-C1 and V3-C2.

The honest risk on V3-C2: even with all ~48 dispatched, per-coefficient promotion likely clears for only the top handful of patterns. If V3-C2 cannot be answered at adequate n , it should be reported as a measured failure boundary rather than held open indefinitely — the same discipline V1 applied to the spectral criterion.

6.3 Instrumentation V3 requires (unrecoverable if missed)

1. **Repair system-as-of on the evidence route, or reject it explicitly** (§2.3, Defect 1; `futonlb/holes/DEFECT-bitemporal-as-of-two-routes.md`). A parameter that silently returns present-time data is worse than one that errors. Until repaired, no as-of-based claim.
2. **Per-dispatch corpus/graph stamps** — a revision number and index `index-as-of` per dispatch would make Experiment 0 reconstructible independently of whether step 4 above succeeds. Cheap belt-and-braces.
3. **Restructure the recall budget — and do not size it to a round number.** Per-subject queries against a 30 s *total* budget cannot work at 9–16 s per query. Sized against claude-9’s post-restart measurements (9.1–16.2 s, mean 14.2 s) and `subjects_for` yielding up to 12 subjects:

common subjects	est. total	clears 120 s?
2	~28 s	yes
6	~86 s	yes
8	~114 s	yes
12	~171 s	no

So a 120 s budget would clear ~8 common subjects and still fail the worst rows — which are precisely the analysis-heavy ones the bias already falls hardest on. **The cheaper structural fix is to cap or rank `subjects_for` by term rarity**, since common terms

cost the most and discriminate least; raising the budget alone treats the symptom. Either way, size from the measurement (~300 s if raising), not from a round number.

4. **A load-bearing outcome field**, distinct from used-ids (§3.3). Without it every use-rate V3 reports overstates, and the e9d008be case shows the gap is real rather than theoretical.
5. **A second runner model**. All 129 rejection-reasons come from codex runners. V3’s rejection taxonomy is *codex*’s behaviour until at least one other model is run against the same corpus. This is the cheapest available generality check and it needs no new problems — the same rows can be re-dispatched to a different runner.
6. **Corpus beyond APM**. With ~48 problems of headroom, V3-C2 needs a different source of witnessed outcomes to reach per-coefficient promotion at any breadth. Identifying that source is a Joe-level scoping decision and is the single biggest open question for V3.

6.4 The War Machine as a non-APM source — thin, and thin in a specific way

Joe, 2026-07-31: *there should be a non-APM source via the War Machine, which can store memories, “although thin”. Assessed below against what V3 actually needs.*

What exists. Measured on a bounded 200-row attachment sample:

domain	attachments	:independently-witnessed	:self-asserted
:mathematics	172	125	38 (+9 no field)
:war-machine	7	6	1

The machinery is wired, not hypothetical. `wm_memory.clj:67` record-episode! writes WM episodes under `:domain :war-machine`; `dark-candidate-projection (:85)` recalls by end-point and emits a proper use-receipt with `surfaced-memory-ids` and `used-memory-ids`; and `wm/independent-phase4-checker` writes an external check (`{:outcome :pass, :witness-status :independently-witnessed, :checker "phase4 dark projection review"}`), tagged [`:war-machine :external-check`]).

Three gaps decide whether it is usable, and none of them is volume.

1. **used-ids means something different here**. In WM it is derived from (`:candidates projection`) — *what the projection algorithm selected*. In the APM lane it is the solver’s report of what it used. These are different signals: one is the system scoring itself, the other is an agent’s attribution. **They must not be pooled into one corpus**. Doing so would be precisely the attribution/witness conflation that V1 §2.3 exists to prevent, committed by us rather than diagnosed by us.
2. **The projection is dark**. `:status :dark, :live-ordering-changed? false`, and the docstring calls it “a detached audit product”. The retrieval does not influence live WM behaviour, so any outcome it did not influence cannot validate it. For V3-C2 the WM path has to go live first.
3. **Witness and memory-use are unlinked**. The phase-4 check carries an outcome and a *mission* subject but **no :memory-use block**, while the use-receipt is keyed by `decision-id / session-id`. There is no join key between them, so no (offered → used → witnessed) triple can be formed — which is the exact shape V3-C2 consumes.

Recommendation. WM is **not** a V3-C2 corpus at present, and writing more WM memories would not make it one — the blockers are the join key and the dark path, not the count. The enabling changes, in order: (a) add a join key between the use-receipt and the external-check record; (b) take the projection live so outcomes are caused by the retrieval. Only then does volume matter.

6.4.1 Correction: the correct implementation is neither lane — it already exists

An earlier draft of this section claimed WM was “the reference implementation of :witness-status done right”, reading the 6-of-7 promotion split as evidence that the field carries information. **That was wrong**, and the correction improves the plan.

holes/CODEX-HANDOFF-live-wm-memory-selection-verify.md:139 (2026-07-23) records: “There is no proper review operation. Earlier live attachments were promoted by **manually reposting hyperedges**.” The 7 WM attachments are dated 2026-07-23. So their :independently-witnessed status was **not earned** — it was hand-posted, and the 6/1 split evidences who reposted what, not a working discipline. My stated caveat (that I had not verified the promoted rows carry witness records) resolves in the unfavourable direction.

But the correct implementation does exist, and it is better placed than either lane. memory_lifecycle.clj:160 review-attachment! was built 2026-07-24 — the day after that handoff — with tests at test/futon3c/peripheral/memory_lifecycle_review_test.clj. Its validator (validate-review!, :105–149) enforces:

- review evidence must exist, and be typed :memory with claim-type :observation or :challenge;
- **the reviewer must not be the memory’s author** (:117–122);
- the review’s subject must name the exact memory, and its pattern set must match the attachment exactly;
- provenance and timestamp must be present;
- **an approval must state its supported witness status** (:142–147), and the resulting witness-status is read **from the review evidence body** (:149) — not from a lane, not from the author.

That is precisely the discipline V1 §2.3 found missing. So the situation is:

	default	promotion
WM record-episode!	✓ :self-asserted	✗ (its 7 rows predate the operation; hand-posted)
Codex lane	✗ lane-derived, cannot fail	✗ bypasses review entirely
review-attachment!	—	✓ evidence-backed, author ≠ reviewer enforced

This makes the §6.3 item 1 repair substantially cheaper than a port: it is routing the codex lane through a shared operation that already exists and is already tested, not writing new review logic. It also means the paper’s finding sharpens — the discipline was not merely known to us, it was *built* and then bypassed by the lane that produced the evidence.

What remains unverified: whether the 7 WM rows would pass review-attachment! if re-played through it. They should be re-reviewed rather than trusted.

7. Division of labour and gates

Per the workspace handoff protocol: experiment harnesses are substantial coding and go to Codex with a bell + park; the plan, review, and paper body stay with the Claude owner. Gates for every dispatched slice: clj-kondo on Clojure, check-parens.el on Lisp/Clojure, relevant tests, and a written determinism check on any new audit artifact.

Immediate next action once agreed: V2-3 (rejection-reason taxonomy), because it is the highest-value asset, needs no repairs to land first, and is the section V2’s identity is built on.

8. Decisions taken, and what remains open

Settled by Joe, 2026-07-31

1. **V2 is the artifact shown to Rob**, conditional on it carrying a plan for V3. That plan is §6, written for an external reader. *V2's scope is therefore fixed: it must be self-contained and honest about what it does not establish, because it will be read by someone outside the stack.*
2. **The as-of defect is logged**, at `futon1b/holes/DEFECT-bitemporal-as-of-two-routes.md`. **It is not an XTDB bug** and must not be taken to James Henderson as one — both faults are `futon1b` application-layer, with named source lines (§2.3). The genuinely XTDB-facing questions (history retention depth; valid-time vs system-time semantics for this use case) are *unestablished* and should be asked as questions after Defect 2 is fixed, not reported as bugs on 2026-08-05.
3. **claude-9 owns the recall-budget fix**. Recorded as step 1 of the §6.2 critical path.

Still open

- **A non-APM source of witnessed outcomes** (§6.3 item 6). Joe proposed the War Machine; assessed at §6.4. Verdict: real machinery, 7 attachments, but blocked on a join key and a dark projection rather than on volume — so it is a *candidate* source requiring two specific changes, not an available one. Whether to make those changes is the open decision. If not, this remains the binding constraint on V3-C2.
- **Whether to attempt the projection-guard repair** (§6.2 step 4) before building per-dispatch snapshotting. Recommend yes — it is one file in our own code and may make Experiment 0 reconstructible retroactively.
- **Whether a second runner model is in scope for V3** (§6.3 item 5), which determines whether the rejection taxonomy generalises or stays codex-specific.